

WEEK-5

Day1(16-03-2026)

Problem-1(Task-1):

Models/Employee.cs

```
using System.ComponentModel.DataAnnotations;

namespace CollegePortalMvcApp.Models
{
    public class Employee
    {
        public int Id { get; set; }

        [Required(ErrorMessage = "Employee name is required")]
        [MinLength(3, ErrorMessage = "Name must be at least 3 characters")]
        public string Name { get; set; }

        [Range(10000, 500000, ErrorMessage = "Salary must be between 10000 and 500000")]
        public decimal Salary { get; set; }

        [Required(ErrorMessage = "Email is required")]
        [EmailAddress(ErrorMessage = "Invalid email format")]
        public string Email { get; set; }

        [Required(ErrorMessage = "Joining Date is required")]
        public DateTime JoiningDate { get; set; }
    }
}
```

Controllers/EmployeeController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using CollegePortalMvcApp.Models;
```

```
namespace CollegePortalMvcApp.Controllers
```

```
{
```

```
    public class EmployeeController : Controller
```

```
    {
```

```
        public IActionResult Register()
```

```
        {
```

```
            return View();
```

```
        }
```

```
        [HttpPost]
```

```
        public IActionResult Register(Employee emp)
```

```
        {
```

```
            if (emp.JoiningDate > DateTime.Now)
```

```
            {
```

```
                ModelState.AddModelError("JoiningDate", "Joining date cannot be in the future");
```

```
            }
```

```
            if (ModelState.IsValid)
```

```
            {
```

```
                return View("Confirmation", emp);
```

```
            }
```

```
            return View(emp);
```

```
        }
```

```
    }
```

```
}
```

Views/Employee/Register.cshtml

@model CollegePortalMvcApp.Models.Employee

<h2>Employee Registration</h2>

<form asp-action="Register" method="post">

<div>

<label>Name</label>

<input asp-for="Name" />

</div>

<div>

<label>Salary</label>

<input asp-for="Salary" />

</div>

<div>

<label>Email</label>

<input asp-for="Email" />

</div>

<div>

<label>Joining Date</label>

<input asp-for="JoiningDate" type="date" />

</div>

<button type="submit">Register</button>

</form>

Views/Employee/Confirmation.cshtml

@model CollegePortalMvcApp.Models.Employee

<h2>Employee Registered Successfully</h2>

<p>Id: @Model.Id</p>

<p>Name: @Model.Name</p>

<p>Salary: @Model.Salary</p>

<p>Email: @Model.Email</p>

<p>Joining Date: @Model.JoiningDate</p>

Problem-2(Task-2):

Models/Product.cs

namespace CollegePortalMvcApp.Models

```
{  
    public class Product  
    {  
        public int Id { get; set; }  
  
        public string Name { get; set; }  
  
        public decimal Price { get; set; }  
  
        public int Quantity { get; set; }  
    }  
}
```

Controllers/ProductController.cs

using Microsoft.AspNetCore.Mvc;

```
using CollegePortalMvcApp.Models;
```

```
namespace CollegePortalMvcApp.Controllers
```

```
{  
    public class ProductController : Controller  
    {  
        private static List<Product> products = new List<Product>();  
        public IActionResult Index()  
        {  
            return View(products);  
        }  
        public IActionResult Create()  
        {  
            return View();  
        }  
        [HttpPost]  
        public IActionResult Create(Product product)  
        {  
            products.Add(product);  
            return RedirectToAction("Index");  
        }  
    }  
}
```

Views/Product/Create.cshtml

```
@model CollegePortalMvcApp.Models.Product
```

```
<h2>Add Product</h2>
```

```
<form asp-action="Create" method="post">
```

```
<div>
```

```
<label>Id</label>
```

```
<input asp-for="Id" />
```

```
</div>
```

```
<br>
```

```
<div>
```

```
<label>Name</label>
```

```
<input asp-for="Name" />
```

```
</div>
```

```
<br>
```

```
<div>
```

```
<label>Price</label>
```

```
<input asp-for="Price" />
```

```
</div>
```

```
<br>
```

```
<div>
```

```
<label>Quantity</label>
```

```
<input asp-for="Quantity" />
```

```
</div>
```

```
<br>
```

```
<button type="submit">Add Product</button>
</form>
```

Views/Product/Index.cshtml

```
@model List<CollegePortalMvcApp.Models.Product>
```

```
<h2>Product List</h2>
```

```
<table border="1">
```

```
<tr>
```

```
<th>Id</th>
```

```
<th>Name</th>
```

```
<th>Price</th>
```

```
<th>Quantity</th>
```

```
</tr>
```

```
@foreach (var product in Model)
```

```
{
```

```
<tr>
```

```
<td>@product.Id</td>
```

```
<td>@product.Name</td>
```

```
<td>@product.Price</td>
```

```
<td>@product.Quantity</td>
```

```
</tr>
```

```
}
```

```
</table>
```


Add New Product

Problem-3(Task-3):

Models/Student.cs

```
namespace CollegePortalMvcApp.Models
```

```
{  
    public class Student  
    {  
        public int Id { get; set; }  
  
        public string Name { get; set; }  
  
        public int Age { get; set; }  
  
        public string Email { get; set; }  
    }  
}
```

Controllers/StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using CollegePortalMvcApp.Models;
```

```
namespace CollegePortalMvcApp.Controllers
```

```
{  
    public class StudentController : Controller  
    {  
        private static List<Student> students = new List<Student>
```



```
{  
    new Student{Id=1, Name="Rahul", Age=21, Email="rahul@gmail.com"},  
    new Student{Id=2, Name="Revathi", Age=22, Email="revathi@gmail.com"}  
};
```

```
public IActionResult Index()
```

```
{  
    return View(students);  
}
```

```
public IActionResult Edit(int id)
```

```
{  
    var student = students.FirstOrDefault(s => s.Id == id);  
    return View(student);  
}
```

```
[HttpPost]
```

```
public IActionResult Edit(Student student)
```

```
{  
    var existingStudent = students.FirstOrDefault(s => s.Id == student.Id);  
  
    if (existingStudent != null)  
    {  
        existingStudent.Name = student.Name;  
        existingStudent.Age = student.Age;  
        existingStudent.Email = student.Email;  
    }  
  
    return RedirectToAction("Index");
```

```
    }  
  }  
}
```

Views/Student/Index.cshtml

```
@model List<CollegePortalMvcApp.Models.Student>
```

```
<h2>Student List</h2>
```

```
<table border="1">
```

```
<tr>
```

```
<th>Id</th>
```

```
<th>Name</th>
```

```
<th>Age</th>
```

```
<th>Email</th>
```

```
<th>Action</th>
```

```
</tr>
```

```
@foreach (var student in Model)
```

```
{
```

```
<tr>
```

```
<td>@student.Id</td>
```

```
<td>@student.Name</td>
```

```
<td>@student.Age</td>
```

```
<td>@student.Email</td>
```

```
<td>
```

```
<a href="/Student/Edit/@student.Id">Edit</a>
```

```
</td>
```

```
</tr>
}
</table>
```

-

```
@model CollegePortalMvcApp.Models.Student
```

```
<h2>Edit Student</h2>
```

```
<form asp-action="Edit" method="post">
```

```
<input type="hidden" asp-for="Id" />
```

```
<div>
```

```
<label>Name</label>
```

```
<input asp-for="Name" />
```

```
</div>
```

```
<br>
```

```
<div>
```

```
<label>Age</label>
```

```
<input asp-for="Age" />
```

```
</div>
```

```
<br>
```

```
<div>
```

```
<label>Email</label>
```

```
<input asp-for="Email" />
```

```
</div>
```

```
<br>
```

```
<button type="submit">Update</button>
```

```
</form>
```

Problem-4(Task-4):

Models/Order.cs

```
namespace CollegePortalMvcApp.Models
```

```
{
```

```
    public class Order
```

```
    {
```

```
        public int OrderId { get; set; }
```

```
        public string CustomerName { get; set; }
```

```
        public decimal Amount { get; set; }
```

```
        public bool IsPremiumCustomer { get; set; }
```

```
    }
```

```
}
```

Controllers/OrderController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using CollegePortalMvcApp.Models;
```

```
namespace CollegePortalMvcApp.Controllers
{
    public class OrderController : Controller
    {
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Order order)
        {
            return View("Details", order);
        }
    }
}
```

Views/Order/Create.cshtml

@model CollegePortalMvcApp.Models.Order

<h2>Create Order</h2>

<form asp-action="Create" method="post">

<div>

<label>Order Id</label>

<input asp-for="OrderId" />

</div>


```
<div>
<label>Customer Name</label>
<input asp-for="CustomerName" />
</div>
```

```
<br>
```

```
<div>
<label>Amount</label>
<input asp-for="Amount" />
</div>
```

```
<br>
```

```
<div>
<label>Premium Customer</label>
<input asp-for="IsPremiumCustomer" type="checkbox" />
</div>
```

```
<br>
```

```
<button type="submit">Submit</button>
```

```
</form>
```

Views/Order/Details.cshtml

@model CollegePortalMvcApp.Models.Order

```
<h2>Order Details</h2>
```

```
<p>Order Id: @Model.OrderId</p>
```

```
<p>Customer Name: @Model.CustomerName</p>
```

```
<p>Amount: @Model.Amount</p>
```

```
@if(Model.IsPremiumCustomer)
```

```
{
```

```
    <p style="color:green">Premium Discount Applied 🎉</p>
```

```
}
```

```
@if(Model.Amount > 5000)
```

```
{
```

```
    <p style="color:blue">Eligible for Free Shipping</p>
```

```
}
```

Day-2(17-03-2026)

Problem-1(Task-1):

Models/Employee.cs

```
namespace CollegePortalMvcApp.Models
```

```
{
```

```
    public class Employee
```

```
    {
```

```
        public string Name { get; set; }
```

```
        public int Age { get; set; }
```

```
    }
```

```
}
```

Controllers/EmployeeController.cs

```
using Microsoft.AspNetCore.Mvc;
using CollegePortalMvcApp.Models;

namespace CollegePortalMvcApp.Controllers
{
    public class EmployeeController : Controller
    {
        public IActionResult Create()
        {
            var employees = new List<Employee>
            {
                new Employee(),
                new Employee(),
                new Employee()
            };
            return View(employees);
        }
        [HttpPost]
        public IActionResult Create(List<Employee> employees)
        {
            return View("Result", employees);
        }
    }
}
```

Views/Employee/Create.cshtml

```
@model List<CollegePortalMvcApp.Models.Employee>
```



```
<h2>Enter Employees</h2>
```

```
<form asp-action="Create" method="post">
```

```
@for (int i = 0; i < Model.Count; i++)
```

```
{
```

```
    <div>
```

```
        <h4>Employee @(i + 1)</h4>
```

```
        <label>Name</label>
```

```
        <input asp-for="@Model[i].Name" />
```

```
        <br>
```

```
        <label>Age</label>
```

```
        <input asp-for="@Model[i].Age" />
```

```
        <br><br>
```

```
    </div>
```

```
}
```

```
<button type="submit">Submit</button>
```

```
</form>
```

Views/Employee/Result.cshtml

```
@model List<CollegePortalMvcApp.Models.Employee>
```

```
<h2>Employee List</h2>
```

```
<table border="1">
```

```
<tr>
```

```
<th>Name</th>
```

```
<th>Age</th>
```

```
</tr>
```

```
@foreach (var emp in Model)
```

```
{
```

```
<tr>
```

```
<td>@emp.Name</td>
```

```
<td>@emp.Age</td>
```

```
</tr>
```

```
}
```

```
</table>
```

Problem-2(Task-2):

Models/User.cs

```
namespace Day2PracticeApp.Models
```

```
{
```

```
    public class User
```

```
    {
```

```
        public string Username { get; set; }
```

```
        public string Password { get; set; }
```

```
    }
```

```
}
```

Controllers/UserController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using Day2PracticeApp.Models;
```

```
namespace Day2PracticeApp.Controllers
```

```
{
```

```
    public class UserController : Controller
```

```
    {
```

```
        [HttpGet]
```

```
        public IActionResult Register()
```

```
        {
```

```
            return View();
```

```
        }
```

```
        [HttpPost]
```

```
        public IActionResult Register(User user)
```

```
        {
```

```
            if (user.Username != null && user.Username.ToLower().Contains("admin"))
```

```
            {
```

```
                ModelState.AddModelError("Username", "Username cannot contain 'admin'");
```

```
            }
```

```
            if (user.Password == null || user.Password.Length < 8)
```

```
            {
```

```
                ModelState.AddModelError("Password", "Password must be at least 8 characters");
```

```
            }
```

```
            if (!ModelState.IsValid)
```

```
            {
```

```
                return View(user);
```

```
            }
```

```
        return View("Success", user);
    }
}
}
```

Views/User/Register.cshtml

@model Day2PracticeApp.Models.User

<h2>User Registration</h2>

<form asp-action="Register" method="post">

<div>

<label>Username</label>

<input asp-for="Username" />

</div>

<div>

<label>Password</label>

<input asp-for="Password" type="password" />

</div>

<button type="submit">Register</button>

</form>

Views/User/Success.cshtml

@model Day2PracticeApp.Models.User

<h2>Registration Successful</h2>

<p>Username: @Model.Username</p>

Problem-3(Task-3):

Models/Product.cs

namespace Day2PracticeApp.Models

```
{  
    public class Product  
    {  
        public int Id { get; set; }  
        public string Name { get; set; }  
        public double Price { get; set; }  
    }  
}
```

Controllers/ProductController.cs

using Microsoft.AspNetCore.Mvc;

using Day2PracticeApp.Models;

namespace Day2PracticeApp.Controllers

```
{  
    public class ProductController : Controller  
    {  
        private static List<Product> products = new List<Product>();  
  
        [HttpGet]  
        public IActionResult Create()
```

```

    {
        return View();
    }

[HttpPost]
public IActionResult Create(Product product, string action)
{
    if (action == "preview")
    {
        return View("Preview", product);
    }
    else if (action == "save")
    {
        products.Add(product);
        return RedirectToAction("Index");
    }

    return View();
}

public IActionResult Index()
{
    return View(products);
}
}
}

```

Views/Product/Create.cshtml

@model Day2PracticeApp.Models.Product

```
<h2>Create Product</h2>
```

```
<form asp-action="Create" method="post">
```

```
  <div>
```

```
    <label>Id</label>
```

```
    <input asp-for="Id" />
```

```
  </div>
```

```
  <br>
```

```
  <div>
```

```
    <label>Name</label>
```

```
    <input asp-for="Name" />
```

```
  </div>
```

```
  <br>
```

```
  <div>
```

```
    <label>Price</label>
```

```
    <input asp-for="Price" />
```

```
  </div>
```

```
  <br>
```

```
  <button type="submit" name="action" value="preview">Preview</button>
```

```
  <button type="submit" name="action" value="save">Save</button>
```

```
</form>
```

Views/Product/Preview.cshtml

```
@model Day2PracticeApp.Models.Product
```

```
<h2>Preview Product</h2>
```

<p>Id: @Model.Id</p>

<p>Name: @Model.Name</p>

<p>Price: @Model.Price</p>

Back

Views/Product/Index.cshtml

@model List<Day2PracticeApp.Models.Product>

<h2>Saved Products</h2>

<table border="1">

<tr>

<th>Id</th>

<th>Name</th>

<th>Price</th>

</tr>

@foreach (var item in Model)

{

<tr>

<td>@item.Id</td>

<td>@item.Name</td>

<td>@item.Price</td>

</tr>

}

</table>

Problem-4(Task-4):**Models/Product.cs**

```
namespace Day2PracticeApp.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
    }
}
```

Controllers/ProductController.cs

```
using Microsoft.AspNetCore.Mvc;
using Day2PracticeApp.Models;
namespace Day2PracticeApp.Controllers
{
    public class ProductController : Controller
    {
        private static List<Product> products = new List<Product>
        {
            new Product { Id = 1, Name = "Laptop", Price = 50000 },
            new Product { Id = 2, Name = "Mobile", Price = 20000 }
        };

        public IActionResult Index()
        {
            return View(products);
        }
    }
}
```

```

public IActionResult Edit(int id)
{
    var product = products.FirstOrDefault(p => p.Id == id);

    return View(product);
}

[HttpPost]
public IActionResult Edit(Product updatedProduct)
{
    var product = products.FirstOrDefault(p => p.Id == updatedProduct.Id);

    if (product != null)
    {
        product.Name = updatedProduct.Name;
        product.Price = updatedProduct.Price;
    }

    return RedirectToAction("Index");
}
}

```

Views/Product/Index.cshtml

```
@model List<Day2PracticeApp.Models.Product>
```

```
<h2>Product List</h2>
```

```
<table border="1">
```

```
<tr>
```

```
<th>Id</th>
```

```

        <th>Name</th>
        <th>Price</th>
        <th>Action</th>
    </tr>

    @foreach (var item in Model)
    {
        <tr>
            <td>@item.Id</td>
            <td>@item.Name</td>
            <td>@item.Price</td>
            <td>
                <a href="/Product/Edit/@item.Id">Edit</a>
            </td>
        </tr>
    }
</table>

```

Views/Product/Edit.cshtml

```
@model Day2PracticeApp.Models.Product
```

```
<h2>Edit Product</h2>
```

```
<form asp-action="Edit" method="post">
```

```
    <input type="hidden" asp-for="Id" />
```

```
    <div>
```

```
        <label>Name</label>
```

```
<input asp-for="Name" />
</div>

<br>

<div>
    <label>Price</label>
    <input asp-for="Price" />
</div>

<br>

<button type="submit">Update</button>

</form>
```

Problem-5(Task-5):

Models/UserProfile.cs

```
namespace Day2PracticeApp.Models
{
    public class UserProfile
    {
        public string Name { get; set; }

        public string Gender { get; set; }

        public List<string> Skills { get; set; }
    }
}
```

Controllers/UserProfileController.cs

```
using Microsoft.AspNetCore.Mvc;
```

```
using Day2PracticeApp.Models;
```

```
namespace Day2PracticeApp.Controllers
```

```
{
```

```
    public class UserProfileController : Controller
```

```
    {
```

```
        public IActionResult Create()
```

```
        {
```

```
            return View();
```

```
        }
```

```
        [HttpPost]
```

```
        public IActionResult Create(UserProfile user)
```

```
        {
```

```
            if (string.IsNullOrEmpty(user.Name))
```

```
            {
```

```
                ModelState.AddModelError("Name", "Name is required");
```

```
            }
```

```
            if (string.IsNullOrEmpty(user.Gender))
```

```
            {
```

```
                ModelState.AddModelError("Gender", "Select gender");
```

```
            }
```

```
            if (user.Skills == null || user.Skills.Count == 0)
```

```
            {
```

```
                ModelState.AddModelError("Skills", "Select at least one skill");
```

```

    }

    if (!ModelState.IsValid)
    {
        return View(user);
    }

    return View("Success", user);
}
}
}

```

Views/UserProfile/Create.cshtml

@model Day2PracticeApp.Models.UserProfile

<h2>User Profile</h2>

<form asp-action="Create" method="post">

<div>

<label>Name</label>

<input asp-for="Name" />

</div>

<div>

<label>Gender</label>

<select asp-for="Gender">

```

        <option value="">--Select--</option>
        <option value="Male">Male</option>
        <option value="Female">Female</option>
    </select>
    <span asp-validation-for="Gender" style="color:red"></span>
</div>

<br>

<div>
    <label>Skills</label><br>

    <input type="checkbox" name="Skills" value="Java"
    @(Model?.Skills != null && Model.Skills.Contains("Java") ? "checked" : "") /> Java

    <br>

    <input type="checkbox" name="Skills" value="C#"
    @(Model?.Skills != null && Model.Skills.Contains("C#") ? "checked" : "") /> C#

    <br>

    <input type="checkbox" name="Skills" value="Python"
    @(Model?.Skills != null && Model.Skills.Contains("Python") ? "checked" : "") /> Python

    <br>

    <span style="color:red">
        @Html.ValidationMessage("Skills")
    </span>
</div>

<br>

<button type="submit">Submit</button>

```

</form>

Views/UserProfile/Success.cshtml

@model Day2PracticeApp.Models.UserProfile

<h2>Form Submitted Successfully 🎉 </h2>

<p>Name: @Model.Name</p>

<p>Gender: @Model.Gender</p>

<p>Skills:</p>

@foreach (var skill in Model.Skills)

{

@skill

}

Day-3(18-03-2026)

Problem-1(Task-1):

Models/Category.cs

using System.ComponentModel.DataAnnotations;

namespace LibraryApp.Models

{

public class Category

{

public int Id { get; set; }

[Required]

public string Name { get; set; }

public ICollection<Book> Books { get; set; }


```
}  
}
```

Models/Book.cs

```
using System.ComponentModel.DataAnnotations;
```

```
namespace LibraryApp.Models
```

```
{  
    public class Book  
    {  
        public int Id { get; set; }  
  
        [Required]  
        public string Title { get; set; }  
  
        [Required]  
        public string Author { get; set; }  
  
        [Range(100, 5000)]  
        public double Price { get; set; }  
  
        public int CategoryId { get; set; }  
  
        public Category Category { get; set; }  
    }  
}
```

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
```

```
using LibraryApp.Models;
```

```
namespace LibraryApp.Data
```

```
{  
    public class AppDbContext : DbContext  
    {  
        public AppDbContext(DbContextOptions<AppDbContext> options)  
            : base(options) { }  
  
        public DbSet<Book> Books { get; set; }  
        public DbSet<Category> Categories { get; set; }  
    }  
}
```

```
Controllers/BookController.cs
```

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using LibraryApp.Data;  
using LibraryApp.Models;  
  
namespace LibraryApp.Controllers  
{  
    public class BookController : Controller  
    {  
        private readonly AppDbContext _context;  
  
        public BookController(AppDbContext context)  
        {  
            _context = context;  
        }  
    }  
}
```

```
}
```

```
public IActionResult Index()
```

```
{
```

```
    var books = _context.Books
```

```
        .Include(b => b.Category) // 🔥 Eager Loading
```

```
        .ToList();
```

```
    ViewBag.TotalBooks = books.Count;
```

```
    ViewBag.MaxPrice = books.Max(b => b.Price);
```

```
    ViewBag.OrderedBooks = books.OrderByDescending(b => b.Price).ToList();
```

```
    return View(books);
```

```
}
```

```
public IActionResult Create()
```

```
{
```

```
    ViewBag.Categories = _context.Categories.ToList();
```

```
    return View();
```

```
}
```

```
[HttpPost]
```

```
public IActionResult Create(Book book)
```

```
{
```

```
    if (ModelState.IsValid)
```

```
    {
```

```
        _context.Books.Add(book);
```

```
        _context.SaveChanges();
```

```
TempData["msg"] = "Book added successfully";
```

```
HttpContext.Session.SetString("LastBook", book.Title);
```

```
return RedirectToAction("Index");
```

```
}
```

```
return View(book);
```

```
}
```

```
public IActionResult Edit(int id)
```

```
{
```

```
    var book = _context.Books.Find(id);
```

```
    ViewBag.Categories = _context.Categories.ToList();
```

```
    return View(book);
```

```
}
```

```
[HttpPost]
```

```
public IActionResult Edit(Book book)
```

```
{
```

```
    _context.Books.Update(book);
```

```
    _context.SaveChanges();
```

```
    TempData["msg"] = "Book updated";
```

```
    return RedirectToAction("Index");
```

```
}
```

```
public IActionResult Delete(int id)
```

```
{
```

```

        var book = _context.Books.Find(id);
        _context.Books.Remove(book);
        _context.SaveChanges();

        TempData["msg"] = "Book deleted";
        return RedirectToAction("Index");
    }

```

```

    public IActionResult ByCategory(int id)
    {
        var books = _context.Books
            .Include(b => b.Category)
            .Where(b => b.CategoryId == id)
            .ToList();

        return View("Index", books);
    }
}

```

Program.cs

```

app.MapControllerRoute(
    name: "booksByCategory",
    pattern: "Books/Category/{id}",
    defaults: new { controller = "Book", action = "ByCategory" });

```

Views/Book/Index.cshtml

```

@model List<LibraryApp.Models.Book>

```

```
<h2>Books</h2>
```

```
@if (TempData["msg"] != null)
```

```
{
```

```
    <p style="color:green">@TempData["msg"]</p>
```

```
}
```

```
<p>Total Books: @ViewBag.TotalBooks</p>
```

```
<p>Max Price: @ViewBag.MaxPrice</p>
```

```
<table border="1">
```

```
<tr>
```

```
    <th>Title</th>
```

```
    <th>Author</th>
```

```
    <th>Price</th>
```

```
    <th>Category</th>
```

```
    <th>Action</th>
```

```
</tr>
```

```
@foreach (var item in Model)
```

```
{
```

```
    <tr>
```

```
        <td>@item.Title</td>
```

```
        <td>@item.Author</td>
```

```
        <td>@item.Price</td>
```

```
        <td>@item.Category?.Name</td>
```

```
        <td>
```

```
            <a href="/Book/Edit/@item.Id">Edit</a> |
```

```
            <a href="/Book/Delete/@item.Id">Delete</a>
```

```
        </td>
    </tr>
}
</table>
```

Problem-2(Task-2):

Models/Product.cs

```
namespace StoreApp.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public double Price { get; set; }
        public int StockQuantity { get; set; }
    }
}
```

Models/Order.cs

```
namespace StoreApp.Models
{
    public class Order
    {
        public int Id { get; set; }
        public DateTime OrderDate { get; set; }
        public double TotalAmount { get; set; }

        public int ProductId { get; set; }
        public Product Product { get; set; }
    }
}
```

```
}  
}
```

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;  
using StoreApp.Models;  
  
namespace StoreApp.Data  
{  
    public class ApplicationDbContext : DbContext  
    {  
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)  
            : base(options) { }  
  
        public DbSet<Product> Products { get; set; }  
        public DbSet<Order> Orders { get; set; }  
    }  
}
```

Program.cs

```
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
options.UseInMemoryDatabase("StoreDB"));  
builder.Services.AddControllersWithViews();
```

Controllers/OrdersController.cs

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.EntityFrameworkCore;  
using StoreApp.Data;  
using StoreApp.Models;
```



```

namespace StoreApp.Controllers
{
    public class OrdersController : Controller
    {
        private readonly AppDbContext _context;

        public OrdersController(AppDbContext context)
        {
            _context = context;
        }

        public IActionResult Index()
        {
            var orders = _context.Orders
                .Include(o => o.Product)
                .AsNoTracking()
                .ToList();

            ViewBag.TotalRevenue = orders.Sum(o => o.TotalAmount);

            ViewBag.LowStock = _context.Products
                .Where(p => p.StockQuantity < 5)
                .ToList();

            ViewBag.Grouped = _context.Orders
                .GroupBy(o => o.Product.Name)
                .Select(g => new
            {

```

```

        Product = g.Key,
        Count = g.Count()
    }).ToList();

    return View(orders);
}

public IActionResult Create()
{
    ViewBag.Products = _context.Products.ToList();
    return View();
}

[HttpPost]
public IActionResult Create(Order order)
{
    var product = _context.Products.Find(order.ProductId);

    if (product != null)
    {
        order.OrderDate = DateTime.Now;
        order.TotalAmount = product.Price;

        product.StockQuantity--;

        _context.Orders.Add(order);
        _context.SaveChanges();

        return RedirectToAction("Index");
    }
}

```

```
}
```

```
ViewBag.Products = _context.Products.ToList();
```

```
return View(order);
```

```
}
```

```
public IActionResult Edit(int id)
```

```
{
```

```
    var order = _context.Orders.Find(id);
```

```
    ViewBag.Products = _context.Products.ToList();
```

```
    return View(order);
```

```
}
```

```
[HttpPost]
```

```
public IActionResult Edit(Order updatedOrder)
```

```
{
```

```
    var oldOrder = _context.Orders
```

```
        .AsNoTracking()
```

```
        .FirstOrDefault(o => o.Id == updatedOrder.Id);
```

```
    var oldProduct = _context.Products.Find(oldOrder.ProductId);
```

```
    var newProduct = _context.Products.Find(updatedOrder.ProductId);
```

```
    oldProduct.StockQuantity++;
```

```
    newProduct.StockQuantity--;
```

```
    updatedOrder.TotalAmount = newProduct.Price;
```

```
    _context.Orders.Update(updatedOrder);
```

```

        _context.SaveChanges();

        return RedirectToAction("Index");
    }

    public IActionResult Details(int id)
    {
        var order = _context.Orders
            .Include(o => o.Product)
            .FirstOrDefault(o => o.Id == id);

        return View(order);
    }
}

```

Views/Orders/Create.cshtml

@model StoreApp.Models.Order

<h2>Create Order</h2>

<form asp-action="Create" method="post">

<div>

<label>Select Product</label>

<select asp-for="ProductId">

<option value="">--Select Product--</option>

@foreach (var item in ViewBag.Products)

{

```
        <option value="@item.Id">
            @item.Name (Stock: @item.StockQuantity)
        </option>
    }
</select>
</div>
```

```
<br>
```

```
<button type="submit">Place Order</button>
```

```
</form>
```

Views/Orders/Index.cshtml

```
@model List<StoreApp.Models.Order>
```

```
<h2>Orders List</h2>
```

```
<p><strong>Total Revenue:</strong> @ViewBag.TotalRevenue</p>
```

```
<h3>Low Stock Products</h3>
```

```
@if (ViewBag.LowStock != null)
```

```
{
    foreach (var p in ViewBag.LowStock)
    {
        <p>@p.Name (Stock: @p.StockQuantity)</p>
    }
}
```

```
<h3>Orders Grouped by Product</h3>
```

```
@if (ViewBag.Grouped != null)
```

```
{
```

```
    foreach (var g in ViewBag.Grouped)
```

```
    {
```

```
        <p>@g.Product - @g.Count Orders</p>
```

```
    }
```

```
}
```

```
<hr />
```

```
<table border="1">
```

```
<tr>
```

```
    <th>Order Id</th>
```

```
    <th>Product</th>
```

```
    <th>Price</th>
```

```
    <th>Date</th>
```

```
    <th>Actions</th>
```

```
</tr>
```

```
@foreach (var item in Model)
```

```
{
```

```
    <tr>
```

```
        <td>@item.Id</td>
```

```
        <td>@item.Product?.Name</td>
```

```
        <td>@item.TotalAmount</td>
```

```
        <td>@item.OrderDate</td>
```

```
        <td>
```

```
            <a href="/Orders/Details/@item.Id">Details</a> |
```

```
        <a href="/Orders/Edit/@item.Id">Edit</a>
    </td>
</tr>
}
</table>
```

```
<br>
```

```
<a href="/Orders/Create">Create New Order</a>
```

Views/Orders/Edit.cshtml

```
@model StoreApp.Models.Order
```

```
<h2>Edit Order</h2>
```

```
<form asp-action="Edit" method="post">
```

```
    <input type="hidden" asp-for="Id" />
```

```
<div>
```

```
    <label>Select Product</label>
```

```
    <select asp-for="ProductId">
```

```
        @foreach (var item in ViewBag.Products)
```

```
        {
```

```
            <option value="@item.Id"
```

```
                @(item.Id == Model.ProductId ? "selected" : "")>
```

```
                @item.Name (Stock: @item.StockQuantity)
```

```
            </option>
```

```
        }
```

</select>

</div>

<button type="submit">Update Order</button>

</form>

Views/Orders/Details.cshtml

@model StoreApp.Models.Order

<h2>Order Details</h2>

<p>Order Id: @Model.Id</p>

<p>Product: @Model.Product?.Name</p>

<p>Price: @Model.TotalAmount</p>

<p>Order Date: @Model.OrderDate</p>

Back to List

Day5(20-03-2026)

Problem-1(Task-1):

Hello MVC Flow1. Hello MVC Flow

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace YourProjectName.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }
    }
}
```

Views/Home/Index.cshtml

```
<h2>Welcome to ASP.NET Core 8</h2>
```

Problem-2(Task-2):

Passing Data using ViewBag

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace YourProjectName.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            ViewBag.Message = "Hello from Controller using ViewBag!";

            return View();
        }
    }
}
```

```
}  
}
```

Views/Home/Index.cshtml

[<h2>@ViewBag.Message</h2>](#)

Problem-3(Task-3):

Strongly Typed Model

Models/Student.cs

```
namespace YourProjectName.Models  
{  
    public class Student  
    {  
        public int Id { get; set; }  
        public string Name { get; set; }  
        public int Age { get; set; }  
    }  
}
```

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;  
using YourProjectName.Models;  
namespace YourProjectName.Controllers  
{  
    public class HomeController : Controller  
    {
```

```

public IActionResult Index()
{
    Student student = new Student()
    {
        Id = 1,
        Name = "Revathi",
        Age = 22
    };

    return View(student);
}
}

```

Views/Home/Index.cshtml

@model YourProjectName.Models.Student

<h2>Student Details</h2>

<p>Id: @Model.Id</p>

<p>Name: @Model.Name</p>

<p>Age: @Model.Age</p>

Problem-4(Task-4):

Multiple Data to View

Models/Student.cs

namespace YourProjectName.Models

```

{
    public class Student
    {

```

```
        public int Id { get; set; }

        public string Name { get; set; }

        public int Age { get; set; }
    }
}
```

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using YourProjectName.Models;
using System.Collections.Generic;
namespace YourProjectName.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            List<Student> students = new List<Student>()
            {
                new Student { Id = 1, Name = "Revathi", Age = 22 },
                new Student { Id = 2, Name = "Ravi", Age = 23 },
                new Student { Id = 3, Name = "Anjali", Age = 21 }
            };

            return View(students);
        }
    }
}
```

Views/Home/Index.cshtml

```
@model List<YourProjectName.Models.Student>
```

```
<h2>Student List</h2>
```

```
<table border="1">
```

```
  <tr>
```

```
    <th>Id</th>
```

```
    <th>Name</th>
```

```
    <th>Age</th>
```

```
  </tr>
```

```
  @foreach (var student in Model)
```

```
  {
```

```
    <tr>
```

```
      <td>@student.Id</td>
```

```
      <td>@student.Name</td>
```

```
      <td>@student.Age</td>
```

```
    </tr>
```

```
  }
```

```
</table>
```

Problem-5(Task-5):

Razor Conditions

Models/Student.cs

```
namespace YourProjectName.Models
```

```
{
```

```
  public class Student
```

```
  {
```

```
    public int Id { get; set; }
```

```
    public string Name { get; set; }
```

```
        public int Age { get; set; }  
    }  
}
```

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;  
using YourProjectName.Models;  
namespace YourProjectName.Controllers  
{  
    public class HomeController : Controller  
    {  
        public IActionResult Index()  
        {  
            Student student = new Student()  
            {  
                Id = 1,  
                Name = "Revathi",  
                Age = 17  
            };  
  
            return View(student);  
        }  
    }  
}
```

Views/Home/Index.cshtml

```
@model YourProjectName.Models.Student  
  
<h2>Student Details</h2>  
  
<p><strong>Name:</strong> @Model.Name</p>
```

```
<p><strong>Age:</strong> @Model.Age</p>
<p>
    <strong>Status:</strong>
    @if (Model.Age > 18)
    {
        <span>Adult</span>
    }
    else
    {
        <span>Minor</span>
    }
</p>
```

Program-6(Task-6):

Form Handling (GET & POST)

Models/Student.cs

```
namespace YourProjectName.Models
{
    public class Student
    {
        public string Name { get; set; }
        public int Age { get; set; }
    }
}
```

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using YourProjectName.Models;
namespace YourProjectName.Controllers
```

```

{
    public class HomeController : Controller
    {
        [HttpGet]
        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Student student)
        {
            return View("Result", student);
        }
    }
}

```

Views/Home/Create.cshtml

@model YourProjectName.Models.Student

```

<h2>Enter Student Details</h2>
<form asp-action="Create" method="post">
    <label>Name:</label>
    <input type="text" asp-for="Name" />
    <br /><br />

    <label>Age:</label>
    <input type="number" asp-for="Age" />
    <br /><br />

```



```
<button type="submit">Submit</button>
</form>
```

Views/Home/Result.cshtml

```
@model YourProjectName.Models.Student
```

```
<h2>Submitted Data</h2>
```

```
<p><strong>Name:</strong> @Model.Name</p>
```

```
<p><strong>Age:</strong> @Model.Age</p>
```

Program-7(Task-7):

Model Binding

Models/User.cs

```
namespace YourProjectName.Models
{
    public class User
    {
        public string Name { get; set; }
        public int Age { get; set; }
        public string Email { get; set; }
    }
}
```

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using YourProjectName.Models;
namespace YourProjectName.Controllers
{
```

```

public class HomeController : Controller
{
    [HttpGet]
    public IActionResult Register()
    {
        return View();
    }

    [HttpPost]
    public IActionResult Register(User user)
    {
        return View("Result", user);
    }
}

```

Views/Home/Register.cshtml

@model YourProjectName.Models.User

<h2>User Registration</h2>

<form asp-action="Register" method="post">

<label>Name:</label>

<input type="text" asp-for="Name" />

<label>Age:</label>

<input type="number" asp-for="Age" />


```
<label>Email:</label>

<input type="email" asp-for="Email" />

<br /><br />

<button type="submit">Submit</button>

</form>
```

Views/Home/Result.cshtml

```
@model YourProjectName.Models.User

<h2>Registered User</h2>

<p><strong>Name:</strong> @Model.Name</p>

<p><strong>Age:</strong> @Model.Age</p>

<p><strong>Email:</strong> @Model.Email</p>
```

Problem-8(Task-8):

Model Validation

Models/User.cs

```
using System.ComponentModel.DataAnnotations;

namespace YourProjectName.Models
{
    public class User
    {
        [Required(ErrorMessage = "Name is required")]
        public string Name { get; set; }

        [Range(18, 60, ErrorMessage = "Age must be between 18 and 60")]
        public int Age { get; set; }
    }
}
```

```
}
```

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using YourProjectName.Models;
namespace YourProjectName.Controllers
{
    public class HomeController : Controller
    {
        [HttpGet]
        public IActionResult Register()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Register(User user)
        {
            if (ModelState.IsValid)
            {
                return View("Result", user);
            }
            return View(user); // return form with errors
        }
    }
}
```

Views/Home/Register.cshtml

```
@model YourProjectName.Models.User
<h2>User Registration</h2>
<form asp-action="Register" method="post">
```

```

<div>
    <label>Name:</label>
    <input asp-for="Name" />
    <span asp-validation-for="Name" style="color:red"></span>
</div>
<br />
<div>
    <label>Age:</label>
    <input asp-for="Age" />
    <span asp-validation-for="Age" style="color:red"></span>
</div>
<br />
<button type="submit">Submit</button>
</form>
@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

Views/Home/Result.cshtml

```

@model YourProjectName.Models.User
<h2>Valid Data Submitted</h2>
<p><strong>Name:</strong> @Model.Name</p>
<p><strong>Age:</strong> @Model.Age</p>

```

Problem-9(Task-9):

ModelState Check

Models/User.cs

```

using System.ComponentModel.DataAnnotations;
namespace YourProjectName.Models

```

```

{
    public class User
    {
        [Required(ErrorMessage = "Name is required")]
        public string Name { get; set; }

        [Range(18, 60, ErrorMessage = "Age must be between 18 and 60")]
        public int Age { get; set; }
    }
}

```

Controllers/HomeController.cs

```

using Microsoft.AspNetCore.Mvc;
using YourProjectName.Models;
namespace YourProjectName.Controllers
{
    public class HomeController : Controller
    {
        [HttpGet]
        public IActionResult Register()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Register(User user)
        {
            if (!ModelState.IsValid)
            {
                return View(user);            }
        }
    }
}

```

```
        return View("Success", user);
    }
}
}
```

Views/Home/Register.cshtml

@model YourProjectName.Models.User

<h2>User Registration</h2>

<form asp-action="Register" method="post">

<div>

<label>Name:</label>

<input asp-for="Name" />

</div>

<div>

<label>Age:</label>

<input asp-for="Age" />

</div>

<button type="submit">Submit</button>

</form>

@section Scripts {

<partial name="_ValidationScriptsPartial" />

}

Views/Home/Success.cshtml

@model YourProjectName.Models.User

<h2>Form Submitted Successfully</h2>

<p>Name: @Model.Name</p>

<p>Age: @Model.Age</p>

Problem-10(Taxk-10):

Custom Routing

Controllers/ProductController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace YourProjectName.Controllers
{
    public class ProductController : Controller
    {
        public IActionResult Details(int id)
        {
            return Content("Product ID: " + id);
        }
    }
}
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.UseRouting();

app.MapControllerRoute(
    name: "productroute",
    pattern: "product/details/{id}",
    defaults: new { controller = "Product", action = "Details" }
);
```



```
app.MapDefaultControllerRoute();  
app.Run();
```

Problem-11(Task-11):

Attribute Routing

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;  
namespace YourProjectName.Controllers  
{  
    public class HomeController : Controller  
    {  
        [Route("home/about")]  
        public IActionResult About()  
        {  
            return Content("This is About Page");  
        }  
    }  
}
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);  
  
var app = builder.Build();  
  
app.UseRouting();  
  
app.MapControllers();  
  
app.Run();
```

Program-12(Task-12):

Session Usage

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddSession();

var app = builder.Build();

app.UseRouting();

app.UseSession();

app.MapDefaultControllerRoute();

app.Run();
```

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace YourProjectName.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Save(string username)
        {
            HttpContext.Session.SetString("Username", username);

            return RedirectToAction("Display");
        }

        public IActionResult Display()
        {
            var name = HttpContext.Session.GetString("Username");

            ViewBag.Username = name;
        }
    }
}
```

```

        return View();
    }
}

```

Views/Home/Index.cshtml

```

<h2>Enter Username</h2>
<form asp-action="Save" method="post">
    <input type="text" name="username" />
    <button type="submit">Save</button>
</form>

```

Views/Home/Display.cshtml

```

<h2>Welcome</h2>
<p>Username: @ViewBag.Username</p>

```

Program-13(Task-13):

TempData Usage

Controllers/HomeController.cs

```

using Microsoft.AspNetCore.Mvc;

namespace YourProjectName.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Submit()

```

```

{
    TempData["SuccessMessage"] = "Form submitted successfully!";

    return RedirectToAction("Result");
}

public IActionResult Result()
{
    return View();
}
}
}

```

Views/Home/Index.cshtml

```

<h2>Submit Form</h2>

<form asp-action="Submit" method="post">
    <button type="submit">Submit</button>
</form>

```

Views/Home/Result.cshtml

```

<h2>Result Page</h2>

@if (TempData["SuccessMessage"] != null)
{
    <p style="color:green">@TempData["SuccessMessage"]</p>
}

```

Program-14(Task-14):

Cookies

Controllers/HomeController.cs

```

using Microsoft.AspNetCore.Mvc;

```

```

namespace YourProjectName.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult SetTheme(string theme)
        {
            CookieOptions options = new CookieOptions()
            {
                Expires = DateTime.Now.AddDays(7)
            };
            Response.Cookies.Append("UserTheme", theme, options);
            return RedirectToAction("Index");
        }
        public IActionResult Index()
        {
            var theme = Request.Cookies["UserTheme"];
            ViewBag.Theme = theme;
            return View();
        }
    }
}

```

Views/Home/Index.cshtml

```

<h2>Select Theme</h2>
<a href="/Home/SetTheme?theme=dark">Dark Mode</a> |
<a href="/Home/SetTheme?theme=light">Light Mode</a>
<br /><br />
<p>Selected Theme: @ViewBag.Theme</p>

```

Program-15(Task-15):

Dependency Injection

Services/GreetingService.cs

```
namespace YourProjectName.Services
{
    public class GreetingService
    {
        public string GetMessage()
        {
            return "Hello from Greeting Service!";
        }
    }
}
```

Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews();
builder.Services.AddScoped<GreetingService>();
var app = builder.Build();
app.UseRouting();
app.MapDefaultControllerRoute();
app.Run();
```

Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc;
using YourProjectName.Services;
namespace YourProjectName.Controllers
{
    public class HomeController : Controller
```

```

{
    private readonly GreetingService _greetingService;

    public HomeController(GreetingService greetingService)
    {
        _greetingService = greetingService;
    }

    public IActionResult Index()
    {
        var message = _greetingService.GetMessage();

        ViewBag.Message = message;

        return View();
    }
}

```

Views/Home/Index.cshtml

[<h2>@ViewBag.Message</h2>](#)

Program-16(Task-16):

Setup EF Core (DbContext)

Data/AppDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using YourProjectName.Models;
namespace YourProjectName.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }
    }
}

```

```

    {
    }

    public DbSet<Student> Students { get; set; }
}
}

```

appsettings.json

```

{
  "ConnectionStrings": {
    "DefaultConnection":
"Server=localhost;Database=StudentDB;Trusted_Connection=True;TrustServerCertificate=True;"
  }
}

```

Program.cs

```

using Microsoft.EntityFrameworkCore;
using YourProjectName.Data;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<AppDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")))
);

var app = builder.Build();

app.UseRouting();

app.MapDefaultControllerRoute();

app.Run();

```

Program-17(Task-17):

Create Table

Models/Student.cs

```
using System.ComponentModel.DataAnnotations;

namespace YourProjectName.Models
{
    public class Student
    {
        public int Id { get; set; }

        [Required]
        public string Name { get; set; }

        public int Age { get; set; }
    }
}
```

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using YourProjectName.Models;

namespace YourProjectName.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options)
        {
        }

        public DbSet<Student> Students { get; set; }
    }
}
```

Migrations/xxxx_CreateStudentTable.cs

```
migrationBuilder.CreateTable(
    name: "Students",
    columns: table => new
    {
        Id = table.Column<int>(nullable: false)
            .Annotation("SqlServer:Identity", "1, 1"),
        Name = table.Column<string>(nullable: false),
        Age = table.Column<int>(nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Students", x => x.Id);
    });
```

Program-18(Task-18):

Insert Data (Create):

Models/Student.cs

```
using System.ComponentModel.DataAnnotations;
namespace YourProjectName.Models
{
    public class Student
    {
        public int Id { get; set; }
        [Required]
        public string Name { get; set; }
        public int Age { get; set; }
    }
}
```

Controllers/StudentController.cs

```
using Microsoft.AspNetCore.Mvc;
using YourProjectName.Data;
using YourProjectName.Models;
namespace YourProjectName.Controllers
{
    public class StudentController : Controller
    {
        private readonly ApplicationDbContext _context;

        public StudentController(ApplicationDbContext context)
        {
            _context = context;
        }

        public IActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Create(Student student)
        {
            if (ModelState.IsValid)
            {
                _context.Students.Add(student);
                _context.SaveChanges();
                return RedirectToAction("Index");
            }

            return View(student);
        }
    }
}
```

```
    }  
  }  
}
```

Views/Student/Create.cshtml

@model YourProjectName.Models.Student

<h2>Add Student</h2>

<form asp-action="Create" method="post">

<div>

<label>Name:</label>

<input asp-for="Name" />

</div>

<div>

<label>Age:</label>

<input asp-for="Age" />

</div>

<button type="submit">Save</button>

</form>

Problem-19(Task-19):

Read Data (LINQ)

Controllers/StudentController.cs

using Microsoft.AspNetCore.Mvc;

using YourProjectName.Data;

using YourProjectName.Models;

using System.Linq;

```

namespace YourProjectName.Controllers
{
    public class StudentController : Controller
    {
        private readonly AppDbContext _context;

        public StudentController(AppDbContext context)
        {
            _context = context;
        }

        public IActionResult Index()
        {
            var students = _context.Students.ToList(); // ✅ LINQ query
            return View(students);
        }
    }
}

```

Views/Student/Index.cshtml

```

@model List<YourProjectName.Models.Student>

<h2>Student List</h2>

<table border="1">
    <tr>
        <th>Id</th>
        <th>Name</th>
        <th>Age</th>
    </tr>
    @foreach (var student in Model)
    {
        <tr>

```

```
        <td>@student.Id</td>

        <td>@student.Name</td>

        <td>@student.Age</td>

    </tr>

}

</table>
```

Problem-20(Task-20):

Update & Delete (CRUD)

Controllers/StudentController.cs

```
public IActionResult Edit(int id)
{
    var student = _context.Students.Find(id);
    if (student == null)
        return NotFound();

    return View(student);
}

[HttpPost]
public IActionResult Edit(Student student)
{
    if (ModelState.IsValid)
    {
        _context.Students.Update(student);
        _context.SaveChanges();

        return RedirectToAction("Index");
    }
}
```

```
    return View(student);  
}
```

Views/Student/Edit.cshtml

```
@model YourProjectName.Models.Student  
  
<h2>Edit Student</h2>  
  
<form asp-action="Edit" method="post">  
    <input type="hidden" asp-for="Id" />  
    <div>  
        <label>Name:</label>  
        <input asp-for="Name" />  
    </div>  
  
    <br />  
    <div>  
        <label>Age:</label>  
        <input asp-for="Age" />  
    </div>  
  
    <br />  
    <button type="submit">Update</button>  
</form>
```

Controllers/StudentController.cs

```
public IActionResult Delete(int id)  
{  
    var student = _context.Students.Find(id);  
    if (student == null)  
        return NotFound();  
    return View(student);  
}
```

```

}

[HttpPost, ActionName("Delete")]
public IActionResult DeleteConfirmed(int id)
{
    var student = _context.Students.Find(id);
    if (student != null)
    {
        _context.Students.Remove(student); // ✅ Delete
        _context.SaveChanges();           // ✅ Save
    }
    return RedirectToAction("Index");
}

```

Views/Student/Delete.cshtml

```

@model YourProjectName.Models.Student

<h2>Delete Student</h2>

<p>Are you sure you want to delete this student?</p>

<p><strong>@Model.Name</strong> (Age: @Model.Age)</p>

<form asp-action="Delete" method="post">
    <input type="hidden" asp-for="Id" />
    <button type="submit">Delete</button>
</form>

```

Views/Student/Index.cshtml

```

<td>

    <a asp-action="Edit" asp-route-id="@student.Id">Edit</a> |

    <a asp-action="Delete" asp-route-id="@student.Id">Delete</a>

</td>

```


